Energy Analytics Visualization Tool

Source Code Documentation

Harshavardhan Dharman, Surya Kannan,
Shashank Venkatesha, Leela Karthikeyan Haribabu

Eindhoven University of Technology - Visualization Course (January 2026)

1 How to Run the Tool

1.1 Requirements

- Python 3.8 or higher
- pip package manager

1.2 Installation

```
pip install -r requirements.txt
```

Or install dependencies manually:

```
pip install dash plotly pandas numpy pycountry pycountry-convert scikit-learn
```

1.3 Running the Application

```
python cia_v1.py
```

The tool will start at `http://127.0.0.1:8050`

1.4 Required Data Files

Place these CSV files in the `Datasets/` folder:

- `energyAnalyst.csv` – Main merged energy dataset (primary)
- `geography_data.csv` – Geographic attributes (Area, Coastline, Land Use)
- `government_and_civics_data.csv` – Government type information
- `energy_data.csv` – Raw energy metrics
- `economy_data.csv` – Economic indicators (GDP, Trade)
- `demographics_data.csv` – Population data
- `transportation_data.csv` – Pipeline infrastructure
- `communications_data.csv` – Communications metrics

2 What We Implemented Ourselves

The following components were developed entirely by our team:

Component	Description
Data Cleaning Pipeline	Custom regex-based normalization of CIA dataset values (units, currencies, missing values) in `clean_numeric_column()` function
Country Name Matching	Manual mapping table (`CIA_NAME_FIXES`) resolving 30+ naming discrepancies between CIA and Gapminder databases
ISO Code Resolution	Fallback lookup using pycountry with fuzzy matching and manual overrides for edge cases
Coordinated Multiple Views	Bidirectional click/brush synchronization across 6+ visualizations using Dash callbacks and `dcc.Store` state management
Parallel Coordinates Brushing	Custom constraint extraction from `restyleData` events and linked table filtering logic
PCA + K-Means Integration	Custom wrapper around sklearn with dynamic parameter selection and cluster coloring
Dark Theme UI	Custom CSS for scrollbars, dropdowns, panels, hover effects, and animations
State Management	Cross-callback synchronization using `dcc.Store` for selections, brushes, and region filters

3 External Libraries Used

Library	Purpose
Dash	Web application framework for building the interactive UI
Plotly	Visualization rendering (choropleth, scatter, bar, parallel coordinates, PCA plots)
Pandas	Data manipulation and CSV loading
NumPy	Numeric operations and array handling
scikit-learn	PCA dimensionality reduction and K-Means clustering algorithms
pycountry	ISO alpha-3 country code lookups
pycountry-convert	Continent inference from country codes
gapminder	Country name standardization reference

4 Project Structure

```
Vis/
```

```
+-- cia_v1.py                  # Main application (1700+ lines)
+-- ReadMe.md                  # Markdown documentation
+-- requirements.txt           # Python dependencies
+-- Datasets/                  # 9 CSV data files
|   +-- energyAnalyst.csv      # Primary merged dataset
|   +-- geography_data.csv
|   +-- government_and_civics_data.csv
|   +-- energy_data.csv
|   +-- economy_data.csv
|   +-- demographics_data.csv
|   +-- transportation_data.csv
|   +-- communications_data.csv
+-- Report/                    # Academic report files
```

5 Code Quality

- **Comments:** Rich inline comments and section headers throughout
- **Modularity:** 65+ functions extracted for reusability
- **Docstring:** Comprehensive module docstring (lines 1–36) explaining features, usage, and dependencies

6 Features

- **Choropleth Map:** Global energy indicator distributions
- **Scatter Plot:** Bivariate relationships with dynamic X/Y axis
- **Bar Chart Rankings:** Top-N countries for any indicator
- **Correlation Heatmap:** Pairwise correlations between indicators
- **Parallel Coordinates:** Multivariate exploration with brushing
- **PCA & K-Means:** Automated country clustering
- **Bidirectional Highlighting:** Click to highlight across all views
- **Region Filtering:** Filter by continent